

```
A&& get_by_rvalue_ref() { ... } // This unnamed rvalue reference is an xvalue.
```

!i&!m

The kind of value that doesn't have identity and isn't movable is the one combination that we haven't yet discussed. But we can disregard it, because that category isn't a useful idea in the C++ language.

Reference-collapsing rules

Multiple like references in an expression (an lvalue reference to an lvalue reference, or an rvalue reference to an rvalue reference) cancel one another out.

- `A& &` collapses into `A&`.
- `A&& &&` collapses into `A&&`.

Multiple unlike references in an expression collapse to an lvalue reference.

- `A& &&` collapses into `A&`.
- `A&& &` collapses into `A&`.

Forwarding references

This final section contrasts rvalue references, which we've already discussed, with the different concept of a *forwarding reference*. Before the term "forwarding reference" was coined, some folks used the term "universal reference".

C++/WinRT

```
void foo(A&& a) { ... }
```

- `A&&` is an rvalue reference, as we've seen. `const` and `volatile` don't apply to rvalue references.
- `foo` accepts only rvalues of type `A`.
- The reason rvalue references (such as `A&&`) exist is so that you can author an overload that's optimized for the case of a temporary (or other rvalue) being passed.

C++/WinRT

```
template <typename _Ty> void bar(_Ty&& ty) { ... }
```

- `_Ty&&` is a *forwarding reference*. Depending what you pass to `bar`, type `_Ty` could be const/non-const independently of volatile/non-volatile.
- `bar` accepts any lvalue or rvalue of type `_Ty`.
- Passing an lvalue causes the forwarding reference to become `_Ty& &&`, which collapses to the lvalue reference `_Ty&`.
- Passing an rvalue causes the forwarding reference to become the rvalue reference `_Ty&&`.
- The reason forwarding references (such as `_Ty&&`) exist is *not* for optimization, but to take what you pass to them and to forward it on transparently and efficiently. You're likely to encounter a forwarding reference only if you write (or closely study) library code—for example, a factory function that forwards on constructor arguments.

Sources

- [Stroustrup, 2013] B. Stroustrup: The C++ Programming Language, Fourth Edition. Addison-Wesley. 2013.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Compile apps with .NET Native

Article • 10/20/2022

.NET Native is a precompilation technology for building and deploying UWP apps. .NET Native is included with Visual Studio 2015 and later versions. It automatically compiles the release version of UWP apps that are written in managed code (C# or Visual Basic) to native code.

Typically, .NET apps are compiled to intermediate language (IL). At run time, the just-in-time (JIT) compiler translates the IL to native code. In contrast, .NET Native compiles UWP apps directly to native code. For developers, this means:

- Your apps feature the performance of native code. Usually, performance will be superior to code that is first compiled to IL and then compiled to native code by the JIT compiler.
- You can continue to program in C# or Visual Basic.
- You can continue to take advantage of the resources provided by .NET Framework, including its class library, automatic memory management and garbage collection, and exception handling.

For users of your apps, .NET Native offers these advantages:

- Faster execution times for the majority of apps and scenarios.
- Faster startup times for the majority of apps and scenarios.
- Low deployment and update costs.
- Optimized app memory usage.

But .NET Native involves more than a compilation to native code. It transforms the way that .NET Framework apps are built and executed. In particular:

- During precompilation, required portions of .NET Framework are statically linked into your app. This allows the app to run with app-local libraries of .NET Framework, and the compiler to perform global analysis to deliver performance wins. As a result, apps launch consistently faster even after .NET Framework updates.
- The .NET Native runtime is optimized for static precompilation and in the vast majority of cases offers superior performance. At the same time, it retains the core reflection features that developers find so productive.

- .NET Native uses the same back end as the C++ compiler, which is optimized for static precompilation scenarios.

.NET Native is able to bring the performance benefits of C++ to managed code developers because it uses the same or similar tools as C++ under the hood, as shown in this table.

Component	.NET Native	C++
Libraries	.NET Framework + Windows Runtime	Win32 + Windows Runtime
Compiler	UTC optimizing compiler	UTC optimizing compiler
Deployed	Ready-to-run binaries	Ready-to-run binaries (ASM)
Runtime	MRT.dll (Minimal CLR Runtime)	CRT.dll (C Runtime)

For UWP apps, you upload .NET Native Code Compilation binaries in app packages (.msix or .appx files) to the Microsoft Store.

.NET Native and compilation

Article • 10/20/2022

Windows desktop applications that target .NET Framework are written in a particular programming language and compiled into intermediate language (IL). At run time, a just-in-time (JIT) compiler is responsible for compiling the IL into native code for the local machine just before a method is executed for the first time. In contrast, the .NET Native tool chain converts source code to native code at compile time. This article compares .NET Native with other compilation technologies available for .NET Framework apps, and also provides a practical overview of how .NET Native produces native code that can help you understand why exceptions that occur in code compiled with .NET Native do not occur in JIT-compiled code.

Generating native binaries

An application that targets .NET Framework and that is not compiled by using the .NET Native tool chain consists of the application assembly, which includes the following:

- [Metadata](#) that describes the assembly, its dependencies, the types it contains, and their members. The metadata is used for reflection and late-bound access, and in some cases by compiler and build tools as well.
- Implementation code. This consists of intermediate language (IL) opcodes. At runtime, the just-in-time (JIT) compiler translates it into native code for the target platform.

In addition to the main application assembly, an app requires that the following be present:

- Any additional class libraries or third-party assemblies that are required by your app. These assemblies similarly include metadata that describes the assembly, its types, and their members, as well as the IL that implements all type members.
- The .NET Framework Class Library. This is a collection of assemblies that is installed on the local system with the .NET Framework installation. The assemblies included in the .NET Framework Class Library include a complete set of metadata and implementation code.
- The common language runtime. This is a collection of dynamic link libraries that perform such services as assembly loading, memory management and garbage collection, exception handling, just-in-time compilation, remoting, and interop.

Like the class library, the runtime is installed on the local system as part of the .NET Framework installation.

Note that the entire common language runtime, as well as metadata and IL for all types in application-specific assemblies, third-party assemblies, and system assemblies must be present for the app to execute successfully.

Just-in-time compilation

The input for the .NET Native tool chain is the UWP app built by the C# or Visual Basic compiler. In other words, the .NET Native tool chain begins execution when the language compiler has finished compilation of a UWP app.

Tip

Because the input to .NET Native is the IL and metadata written to managed assemblies, you can still perform custom code generation or other custom operations by using pre-build or post-build events or by modifying the MSBuild project file.

However, categories of tools that modify IL and thereby prevent the .NET tool chain from analyzing an app's IL are not supported. Obfuscators are the most notable tools of this type.

In the course of converting an app from IL to native code, the .NET Native tool chain performs operations like the following:

- For certain code paths, it replaces code that relies on reflection and metadata with static native code. For example, if a value type does not override the [ValueType.Equals](#) method, the default test for equality uses reflection to retrieve [FieldInfo](#) objects that represent the value type's fields, then compares the field values of two instances. When compiling to native code, the .NET Native tool chain replaces the reflection code and metadata with a static comparison of the field values.
- Where possible, it attempts to eliminate all metadata.
- It includes in the final app assemblies only the implementation code that is actually invoked by the app. This particularly affects code in third-party libraries and in the .NET Framework Class Library. As a result, an application no longer depends on either third-party libraries or the full .NET Framework Class Library; instead, code in third-party and .NET Framework class libraries is now local to the app.

- It replaces the full CLR with a refactored runtime that primarily contains the garbage collector. The refactored runtime is found in a library named `mrt100_app.dll` that is local to the app and is only a few hundred kilobytes in size. This is possible because static linking eliminates the need for many of the services performed by the common language runtime.

ⓘ Note

.NET Native uses the same garbage collector as the standard common language runtime. In the .NET Native garbage collector, background garbage collection is enabled by default. For more information about garbage collection, see **Fundamentals of Garbage Collection**.

ⓘ Important

.NET Native compiles an entire application to a native application. It does not allow you to compile a single assembly that contains a class library to native code so that it can be called independently from managed code.

The resulting app that is produced by the .NET Native tool chain is written to a directory named `ilc.out` in the Debug or Release directory of your project directory. It consists of the following files:

- `<appName>.exe`, a stub executable that simply transfers control to a special `Main` export in `<appName>.dll`.
- `<appName>.dll`, a Windows dynamic link library that contains all your application code, as well as code from the .NET Framework Class Library and any third-party libraries that you have a dependency on. It also contains support code, such as the code necessary to interoperate with Windows and to serialize objects in your app.
- `mrt100_app.dll`, a refactored runtime that provides runtime services such as garbage collection.

All dependencies are captured by the app's APPX manifest. In addition to the application `exe`, `dll`, and `mrt100_app.dll`, which are bundled directly in the appx package, this includes two more files:

- `msvcr140_app.dll`, the C run-time (CRT) library used by `mrt100_app.dll`. It is included by a framework reference in the package.

- `mrt100.dll`. This library includes functions that can improve the performance of `mrt100_app.dll`, although its absence does not prevent `mrt100_app.dll` from functioning. It is loaded from the `system32` directory on the local machine, if it is present.

Because the .NET Native tool chain links implementation code into your app only if it knows that your app actually invokes that code, either the metadata or the implementation code required in the following scenarios may not be included with your app:

- Reflection.
- Dynamic or late-bound invocation.
- Serialization and deserialization.
- COM interop.

If the necessary metadata or implementation code is absent at runtime, the .NET Native runtime throws an exception. You can prevent these exceptions, and ensure that the .NET Native tool chain includes the required metadata and implementation code, by using a [runtime directives file](#), an XML file that designates the program elements whose metadata or implementation code must be available at runtime and assigns a runtime policy to them. The following is the default runtime directives file that's added to a UWP project that's compiled by the .NET Native tool chain:

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="*Application*" Dynamic="Required All" />
  </Application>
</Directives>
```

This enables all the types, as well as all their members, in all the assemblies in your app package for reflection and dynamic invocation. However, it does not enable reflection or dynamic activation of types in .NET Framework Class Library assemblies. In many cases, this is adequate.

See also

- [Metadata and self-describing components](#)
- [Reflection and .NET Native](#)
- [.NET Native general troubleshooting](#)

Getting Started with .NET Native

Article • 10/20/2022

Whether you're writing a new UWP app or migrating an existing Windows 8.x app (previously also called a Microsoft Store app), you can follow the same set of procedures. To create a .NET Native app, follow these steps:

1. [Develop a Universal Windows Platform \(UWP\) app](#), and test the debug builds of your app to ensure that it works properly.
2. [Handle additional reflection and serialization usage](#).
3. [Deploy and test the release builds of your app](#).
4. [Manually resolve missing metadata](#), and repeat [step 3](#) until all issues are resolved.

ⓘ Note

If you are migrating an existing Windows 8.x app to .NET Native, be sure to review [Migrating Your Windows 8.x App to .NET Native](#).

Step 1: Develop and test debug builds of your UWP app

Whether you are developing a new app or migrating an existing one, you follow the same process as for any Windows app.

1. Create a new UWP project in Visual Studio by using the Universal Windows app template for Visual C# or Visual Basic. By default, all UWP applications target the CoreCLR and their release builds are compiled by using the .NET Native tool chain.
2. Note that there are some known compatibility issues between compiling UWP app projects with the .NET Native tool chain and without it. Refer to the [migration guide](#) for more information.

You can now write C# or Visual Basic code against the .NET Native surface area that runs on the local system (or in the simulator).

ⓘ Important

As you develop your app, note any use of serialization or reflection in your code.

By default, debug builds are JIT-compiled to enable rapid F5 deployment, while release builds are compiled by using the .NET Native pre-compilation technology. This means you should build and test the debug builds of your app to ensure that they work normally before compiling them with the .NET Native tool chain.

Step 2: Handle additional reflection and serialization usage

A runtime directives file, `Default.rd.xml`, is automatically added to your project when you create it. If you develop in C#, it is found in your project's **Properties** folder. If you develop in Visual Basic, it is found in your project's **My Project** folder.

ⓘ Note

For an overview of the .NET Native compilation process that provides background on why a runtime directives file is needed, see [.NET Native and Compilation](#).

The runtime directives file is used to define the metadata that your app needs at run time. In some cases, the default version of the file may be adequate. However, some code that relies on serialization or reflection may require additional entries in the runtime directives file.

Serialization

There are two categories of serializers, and both may require additional entries in the runtime directives file:

- Non-reflection based serializers. The serializers found in the .NET Framework class library, such as the [DataContractSerializer](#), [DataContractJsonSerializer](#), and [XmlSerializer](#) classes, do not rely on reflection. However, they do require that code be generated based on the object to be serialized or deserialized. For more information, see the "Microsoft Serializers" section in [Serialization and Metadata](#).
- Third-party serializers. Third-party serialization libraries, the most common of which is the Newtonsoft JSON serializer, are generally reflection-based and require entries in the *.rd.xml file to support object serialization and deserialization. For more information, see the "Third-Party Serializers" section in [Serialization and Metadata](#).

Methods that rely on reflection

In some cases, the use of reflection in code is not obvious. Some common APIs or programming patterns aren't considered part of the reflection API but rely on reflection to execute successfully. This includes the following type instantiation and method construction methods:

- The [Type.MakeGenericType](#) method
- The [Array.CreateInstance](#) and [Type.MakeArrayType](#) methods
- The [MethodInfo.MakeGenericMethod](#) method.

For more information, see [APIs That Rely on Reflection](#).

⚠ Note

Type names used in runtime directives files must be fully qualified. For example, the file must specify "System.String" instead of "String".

Step 3: Deploy and test the release builds of your app

After you've updated the runtime directives file, you can rebuild and deploy release builds of your app. .NET Native binaries are placed in the ILC.out subdirectory of the directory specified in the **Build output path** text box of the project's **Properties** dialog box, **Compile** tab. Binaries that aren't in this folder haven't been compiled with .NET Native. Test your app thoroughly, and test all scenarios, including failure scenarios, on each of its target platforms.

If your app doesn't work properly (particularly in cases where it throws [MissingMetadataException](#) or [MissingInteropDataException](#) exceptions at run time), follow the instructions in the next section, [Step 4: Manually resolve missing metadata](#). Enabling first-chance exceptions may help you find these bugs.

When you've tested and debugged the debug builds of your app and you're confident that you've eliminated the [MissingMetadataException](#) and [MissingInteropDataException](#) exceptions, you should test your app as an optimized .NET Native app. To do this, change your active project configuration from **Debug** to **Release**.

Step 4: Manually resolve missing metadata

The most common failure you'll encounter with .NET Native that you don't encounter on the desktop is a runtime [MissingMetadataException](#), [MissingInteropDataException](#), or [MissingRuntimeArtifactException](#) exception. In some cases, the absence of metadata can manifest itself in unpredictable behavior or even in app failures. This section discusses how you can debug and resolve these exceptions by adding directives to the runtime directives file. For information about the format of runtime directives, see [Runtime Directives \(rd.xml\) Configuration File Reference](#). After you've added runtime directives, you should [deploy and test your app](#) again and resolve any new [MissingMetadataException](#), [MissingInteropDataException](#), and [MissingRuntimeArtifactException](#) exceptions until you encounter no more exceptions.

Tip

Specify the runtime directives at a high level to enable your app to be resilient to code changes. We recommend adding runtime directives at the namespace and type levels rather than the member level. Note that there may be a tradeoff between resiliency and larger binaries with longer compile times.

When addressing a missing metadata exception, consider these issues:

- What was the app trying to do before the exception?
 - For example, was it data binding, serializing or deserializing data, or directly using the reflection API?
- Is this an isolated case, or do you believe you'll encounter the same issue for other types?
 - For example, a [MissingMetadataException](#) exception is thrown when serializing a type in the app's object model. If you know other types that will be serialized, you can add runtime directives for those types (or for their containing namespaces, depending on how well the code is organized) at the same time.
- Can you rewrite the code so it doesn't use reflection?
 - For example, does the code use the `dynamic` keyword when you know what type to expect?
 - Does the code call a method that depends on reflection when some better alternative is available?

Note

For additional information about handling problems that stem from differences in reflection and the availability of metadata in desktop apps and .NET Native, see [APIs That Rely on Reflection](#).

For some specific examples of handling exceptions and other issues that occur when testing your app, see:

- [Example: Handling Exceptions When Binding Data](#)
- [Example: Troubleshooting Dynamic Programming](#)
- [Runtime Exceptions in .NET Native Apps](#)

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [.NET Native and Compilation](#)
- [Reflection and .NET Native](#)
- [APIs That Rely on Reflection](#)
- [Serialization and Metadata](#)
- [Migrating Your Windows 8.x App to .NET Native](#)

Example: Troubleshooting Dynamic Programming

Article • 10/20/2022

🚨 Note

This topic refers to the .NET Native Developer Preview, which is pre-release software. You can download the preview from the [Microsoft Connect website](#) (requires registration).

Not all metadata lookup failures in apps developed using the .NET Native tool chain result in an exception. Some can manifest in unpredictable ways in an app. The following example shows an access violation caused by referencing a null object:

Output

```
Access violation - code c0000005 (first chance)
App!$3_App::Core::Util::NavigationArgs.Setup
App!$3_App::Core::Util::NavigationArgs..ctor
App!$0_App::Gibbon::Util::DesktopNavigationArgs..ctor
App!$0_App::ViewModels::DesktopAppVM.NavigateToPage
App!$3_App::Core::ViewModels::AppViewModel.NavigateToFirstPage
App!$3_App::Core::ViewModels::AppViewModel::<HandleLaunch>d__a.MoveNext
App!$43_System::Runtime::CompilerServices::AsyncMethodBuilderCore.CallMoveNe
xt
App!System::Action.InvokeClosedStaticThunk
App!System::Action.Invoke
App!$43_System::Threading::Tasks::AwaitTaskContinuation.InvokeAction
App!$43_System::Threading::SendOrPostCallback.InvokeOpenStaticThunk
[snip]
```

Let's try to troubleshoot this exception by using the three-step approach outlined in the "Manually resolve missing metadata" section of [Getting Started](#).

What was the app doing?

The first thing to note is the `async` keyword machinery at the base of the stack. Determining what the app was really doing in an `async` method can be problematic, because the stack has lost the context of the originating call and has run the `async` code on a different thread. However, we can deduce that the app is trying to load its first

page. In the implementation for `NavigationArgs.Setup`, the following code caused the access violation:

```
AppViewModel.Current.LayoutVM.PageMap
```

In this instance, the `LayoutVM` property on `AppViewModel.Current` was `null`. Some absence of metadata caused a subtle behavior difference and resulted in a property being uninitialized instead of set, as the app expected. Setting a breakpoint in the code where `LayoutVM` should have been initialized might throw light on the situation. However, note that `LayoutVM`'s type is `App.Core.ViewModels.Layout.LayoutApplicationVM`. The only metadata directive present so far in the rd.xml file is:

XML

```
<Namespace Name="App.ViewModels" Browse="Required Public" Dynamic="Required Public" />
```

A likely candidate for the failure is that

`App.Core.ViewModels.Layout.LayoutApplicationVM` is missing metadata because it's in a different namespace.

In this case, adding a runtime directive for `App.Core.ViewModels` resolved the issue. The root cause was an API call to the `Type.GetType(String)` method that returned `null`, and the app silently ignored the problem until a crash occurred.

In dynamic programming, a good practice when using reflection APIs under .NET Native is to use the `Type.GetType` overloads that throw an exception on failure.

Is this an isolated case?

Other issues might also arise when using `App.Core.ViewModels`. You must decide whether it's worth identifying and fixing each missing metadata exception, or saving time and adding directives for a larger class of types. Here, adding `dynamic` metadata for `App.Core.ViewModels` might be the best approach if the resulting size increase of the output binary isn't an issue.

Could the code be rewritten?

If the app had used `typeof(LayoutApplicationVM)` instead of

`Type.GetType("LayoutApplicationVM")`, the tool chain could have preserved `browse` metadata. However, it still wouldn't have created `invoke` metadata, which would have

led to a [MissingMetadataException](#) exception when instantiating the type. To prevent the exception, you'd still have to add a runtime directive for the namespace or the type that specifies the `dynamic` policy. For information on runtime directives, see the [Runtime Directives \(rd.xml\) Configuration File Reference](#).

See also

- [Getting Started](#)
- [Example: Handling Exceptions When Binding Data](#)

Example: Handling Exceptions When Binding Data

Article • 10/20/2022

The following example shows how to resolve a [MissingMetadataException](#) exception that is thrown when an app compiled with the .NET Native tool chain tries to bind data. Here's the exception information:

Output

```
This operation cannot be carried out as metadata for the following type was removed for performance reasons:  
App.ViewModels.MainPageVM
```

Here's the associated call stack:

Output

```
Reflection::Execution::ReflectionDomainSetupImplementation.CreateNonInvokabilityException+0x238  
Reflection::Core::ReflectionDomain.CreateNonInvokabilityException+0x2e  
Reflection::Core::Execution::ExecutionEnvironment.+0x316  
System::Reflection::Runtime::PropertyInfos::RuntimePropertyInfo.GetValue+0x1cb  
System::Reflection::PropertyInfo.GetValue+0x22  
System::Runtime::InteropServices::WindowsRuntime::CustomPropertyImpl.GetValue+0x42  
App!$66_Interop::McgNative.Func_IInspectable_IInspectable+0x158  
App!$66_Interop::McgNative::__vtable_Windows_UI_Xaml_Data__ICustomProperty.GetValue__STUB+0x46  
Windows_UI_Xaml!DirectUI::PropertyProviderPropertyAccess::GetValue+0x3f  
Windows_UI_Xaml!DirectUI::PropertyAccessPathStep::GetValue+0x31  
Windows_UI_Xaml!DirectUI::PropertyPathListener::ConnectPathStep+0x113
```

What was the app doing?

At the base of the stack, frames from the [Windows.UI.Xaml](#) namespace indicate that the XAML rendering engine was running. The use of the [PropertyInfo.GetValue](#) method indicates a reflection-based lookup of a property's value on the type whose metadata was removed.

The first step in providing a metadata directive would be to add `serialize` metadata for the type so that its properties are all accessible:

XML

```
<Type Name="App.ViewModels.MainPageVM" Serialize="Required Public" />
```

Is this an isolated case?

In this scenario, if data binding has incomplete metadata for one `ViewModel`, it may for others, too. If the code is structured in a way that the app's view models are all in the `App.ViewModels` namespace, you could use a more general runtime directive:

XML

```
<Namespace Name="App.ViewModels " Serialize="Required Public" />
```

Could the code be rewritten to not use reflection?

Because data binding is reflection-intensive, changing the code to avoid reflection isn't feasible.

However, there are ways to specify the `ViewModel` to the XAML page so that the tool chain can associate property bindings with the correct type at compile time and keep the metadata without using a runtime directive. For example, you could apply the [Windows.UI.Xaml.Data.BindableAttribute](#) attribute on properties. This causes the XAML compiler to generate the required lookup information and avoids requiring a runtime directive in the `Default.rd.xml` file.

See also

- [Getting Started](#)
- [Example: Troubleshooting Dynamic Programming](#)

Runtime Exceptions in .NET Native Apps

Article • 10/20/2022

It is important to test the release builds of your Universal Windows Platform app on their target platforms because the debug and release configurations are completely different. By default, the debug configuration uses the .NET Core runtime to compile your app, but the release configuration uses .NET Native to compile your app to native code.

Important

For information on dealing with the [MissingMetadataException](#), [MissingInteropDataException](#), and [MissingRuntimeArtifactException](#) exceptions that you may encounter when testing the release versions of your app, see "Step 4: Manually resolve missing metadata: in the [Getting Started](#) topic, as well as [Reflection and .NET Native](#) and [Runtime Directives \(rd.xml\) Configuration File Reference](#).

Debug and release builds

When the debug build executes against the .NET Core runtime, it has not been compiled to native code. This makes all of the services ordinarily provided by the runtime available to your app.

On the other hand, the release build compiles to native code for its target platforms, removes most dependencies on external runtimes and libraries, and heavily optimizes code for maximum performance.

When you debug release builds that are compiled by using .NET Native:

- You use the .NET Native debug engine, which is different from the normal .NET debugging tools.
- The size of your executable is reduced as much as possible. One of the ways that .NET Native reduces the size of an executable is by significantly trimming runtime exception messages, a topic discussed in greater detail in the [Runtime exception messages](#) section.
- Your code is heavily optimized. This means that inlining is used whenever possible. (Inlining moves code from external routines into the calling routine.) The fact that

.NET Native provides a specialized runtime and implements aggressive inlining affects the call stack that is displayed when debugging. For more information, see the [Runtime call stack](#) section.

ⓘ Note

You can control whether the debug and release builds are compiled with the .NET Native tool chain by checking or unchecking the **Compile with .NET Native tool chain** box. However, Microsoft Store will always compile the production version of your app with the .NET Native tool chain.

Runtime exception messages

To minimize application executable size, .NET Native does not include the full text of exception messages. As a result, runtime exceptions thrown in release builds may not display the full text of exception messages. Instead, the text may consist of a substring along with a link to follow for more information. For example, the exception information may appear as:

Output

```
Exception thrown: '$16_System.AggregateException' in Unknown Module.
```

```
Additional information: AggregateException_ctor_DefaultMessage
```

```
If there is a handler for this exception, the program may be safely continued.
```

If you need the complete exception message, run the debug build instead. For example, the previous exception information from the release build might appear as follows in the debug build:

Output

```
Exception thrown: 'System.AggregateException' in NativeApp.exe.
```

```
Additional information: Value does not fall within the expected range.
```

Runtime call stack

Because of inlining and other optimizations, the call stack displayed by an app compiled by the .NET Native tool chain may not help you to clearly identify the path to a runtime exception.

To get the full stack, run the debug build instead.

See also

- [Debugging .NET Native Windows Universal Apps](#) 
- [Getting Started](#)

Measure startup improvement with .NET Native

Article • 10/20/2022

.NET Native significantly improves the launch time of apps. This improvement is particularly noticeable on portable, low-powered devices and with complex apps. This topic helps you get started with the basic instrumentation needed to measure this startup improvement.

To facilitate performance investigations, .NET Framework and Windows use an event framework called Event Tracing for Windows (ETW) that allows your app to notify tooling when events happen. You can then use a tool called PerfView to easily view and analyze of ETW events. This topic explains how to:

- Use the [EventSource](#) class to emit events.
- Use PerfView to gather those events.
- Use PerfView to display those events.

Using EventSource to emit events

[EventSource](#) provides a base class from which to create a custom event provider. Generally, you create a subclass of [EventSource](#) and wrap the `Write*` methods with your own event methods. A singleton pattern is generally used for each [EventSource](#).

For example, the class in the following example can be used to measure two performance characteristics:

- The time until the `App` class constructor was called.
- The time until the `MainPage` constructor was called.

C#

```
using System;
using System.Diagnostics.Tracing;

[EventSource(Name="MyCompany-MyApp")]
public sealed class AppEventSource : EventSource
{
    public static AppEventSource Log = new AppEventSource ();

    // The numbers passed to WriteEvent and EventAttribute
```

```
// must increment with each logging method.
[Event(1)]
public void AppInitialized() { WriteEvent(1, ""); }

[Event(2)]
public void MainPageInitialized() { WriteEvent(2, ""); }
}
```

There are a few things to notice here. First, a singleton is created in `AppEventSource.Log`. That instance will be used for all logging. Second, each event method has an `EventAttribute`. This helps tooling associate the index of the `WriteEvent` method with the method that was called on `AppEventSource`.

Note that these events are purely illustrative. Most app code will run after these events. You should understand which events in code correspond to user interactions, measure those, and improve those benchmarks. Also, the events themselves log only a single instance in time. It's often useful to have paired start and stop events for every operation. When examining app startup, the start event is generally the "Process/Start" event that the operating system emits.

For example, suppose you are creating an RSS reader. A few interesting locations to log an event are:

- When the main page is first rendered.
- When old RSS stories are deserialized from local storage.
- When your app begins syncing new stories.
- When your app has finished syncing new stories.

Instrumenting an app is straightforward: Just call the appropriate method on the derived class. Using `AppEventSource` from the previous example, you can instrument an app as follows:

```
C#

using System;
using Windows.ApplicationModel;
using Windows.UI.Xaml;

public sealed partial class App
{
    public App()
    {
        this.InitializeComponent();
        this.Suspending += OnSuspending;
        AppEventSource.Log.AppInitialized();
    }
}
```

```

    }
}
public sealed partial class MainPage : Page
{
    public MainPage()
    {
        this.InitializeComponent();
        AppEventSource.Log.MainPageInitialized();
    }
}

```

When the app is instrumented, you're ready to gather events.

Gathering events with PerfView

PerfView uses ETW events to help you do all sorts of performance investigations on your app. It also includes a configuration GUI that lets you turn logging for different types of events on or off. [PerfView](#) is an open-source tool. For more information, watch the PerfView tutorial videos.

ⓘ Note

PerfView cannot be used to collect events on Arm systems. To collect events on Arm systems, use Windows Performance Recorder (WPR). For more information, see [Vance Morrison's blog post](#).

You can also invoke PerfView from the command line. To log only the events from your provider, open the Command Prompt window and enter the command:

Console

```

perfview -KernelEvents:Process -OnlyProviders:*MyCompany-MyApp collect
outputFile

```

where:

`-KernelEvents:Process`

Indicates that you want to know when the process starts and stops. You need the Process/Start event for your app so it can be subtracted from other event times.

`-OnlyProviders:*MyCompany-MyApp`

Turns off other providers that PerfView turns on by default, and turns on the MyCompany-MyApp provider. (The asterisk indicates that it is an [EventSource](#).)

```
collect outputFile
```

Indicates that you want to start collection and save the data to outputFile.etl.zip.

Run your app after starting PerfView. There are a few things to remember when running your app:

- Use a release build, not a debug build. Debug builds often contain extra error checking and error handling code that can cause your app to run slower than expected.
- Running your app with a debugger attached affects the performance of your app.
- Windows uses multiple caching strategies to speed up app launch times. If your app is currently cached in memory and doesn't have to be loaded from disk, it will start faster. To ensure consistency, start and close your app a few times before measuring it.

When you've run your app so that PerfView can collect emitted events, choose the **Stop Collection** button. Generally, you should stop collection before closing your app so you don't get extraneous events. However, if you're measuring shutdown or suspension performance, you'll want to continue collection.

Displaying the events

To view the events that have already been collected, use PerfView to open the .etl or .etl.zip file you created and choose **Events**. ETW will have collected information about a large number of events, including events from other processes. To focus your investigation, complete the following text boxes in the events view:

- In the **Process Filter** box, specify your app name (without ".exe").
- In the **Event Types Filter** box, specify `Process/Start | MyCompany-MyApp`. This sets a filter for events from MyCompany-MyApp and the Windows Kernel/Process/Start event.

Select all the events listed in the left pane (Ctrl-A) and choose the **Enter** key. Now, you should be able to see the timestamps from each event. These timestamps are relative to the start of the trace, so you have to subtract the time of each event from the start time of the process to identify the elapsed time since startup. If you use Ctrl+Click to select two timestamps, you'll see the difference between them displayed in the status bar at

the bottom of the page. This makes it easy to see the elapsed time between any two events in the display (including process start). You can open the shortcut menu for the view and select from a number of useful options, like exporting to CSV files or opening Microsoft Excel to save or process the data.

By repeating the procedure for both your original app and the version you built by using the .NET Native tool chain, you can compare the difference in performance. .NET Native apps generally start faster than non-.NET Native apps. If you're interested in digging deeper, PerfView can also identify the parts of your code that are taking the most time. For more information, watch the PerfView tutorials or read [Vance Morrison's blog entry](#).

See also

- [EventSource](#)

Reflection and .NET Native

Article • 10/20/2022

In .NET Framework, managed development supports metaprogramming through the reflection API. Reflection allows you to inspect objects in an app, call methods on objects discovered through inspection, generate new types at run time, and supports many other dynamic code scenarios. It also supports serialization and deserialization, which allows an object's field values to be persisted and later restored. These scenarios all require the .NET Framework just-in-time (JIT) compiler to generate native code based on available metadata.

The .NET Native runtime doesn't include a JIT compiler. As a result, all necessary native code must be generated in advance. A set of heuristics is used to determine what code should be generated, but these heuristics cannot cover all possible metaprogramming scenarios. Therefore, you must provide hints for these metaprogramming scenarios by using [runtime directives](#). If the necessary metadata or implementation code is not available at runtime, your app throws a [MissingMetadataException](#), [MissingRuntimeArtifactException](#), or [MissingInteropDataException](#) exception. Two troubleshooters are available that will generate the appropriate entry for your runtime directives file that eliminates the exception:

- The [MissingMetadataException troubleshooter](#) [↗](#) for types.
- The [MissingMetadataException troubleshooter](#) [↗](#) for methods.

ⓘ Note

For an overview of the .NET Native compilation process that provides background on why a runtime directives file is needed, see [.NET Native and Compilation](#).

In addition, .NET Native doesn't allow you to reflect over private members of the .NET Framework class library. For example, a call to the [TypeInfo.DeclaredFields](#) property to retrieve the fields of a .NET Framework class library type returns only public or protected fields.

The following articles provide the conceptual and reference documentation that you need to support reflection and serialization in your apps:

- [APIs That Rely on Reflection](#)
- [Reflection API Reference](#)

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)

See also

- [Compiling Apps with .NET Native](#)
- [.NET Native and Compilation](#)

APIs That Rely on Reflection

Article • 10/20/2022

In some cases, the use of reflection in code isn't obvious, and so the .NET Native tool chain doesn't preserve metadata that is needed at run time. This topic covers some common APIs or common programming patterns that aren't considered part of the reflection API but that rely on reflection to execute successfully. If you use them in your source code, you can add information about them to the runtime directives (.rd.xml) file so that calls to these APIs do not throw a [MissingMetadataException](#) exception or some other exception at run time.

Type.MakeGenericType method

You can dynamically instantiate a generic type `AppClass<T>` by calling the [Type.MakeGenericType](#) method by using code like this:

C#

```
var t = Type.GetType("App1.AppClass`1", true);
Type[] typeArgs = {typeof(int)};
Type t2 = t.MakeGenericType(typeArgs);
Activator.CreateInstance(t2);
```

For this code to succeed at run time, several items of metadata are required. The first is `Browse` metadata for the uninstantiated generic type, `AppClass<T>`:

XML

```
<Type Name="App1.AppClass`1" Browse="Required PublicAndInternal" />
```

This allows the [Type.GetType\(String, Boolean\)](#) method call to succeed and return a valid [Type](#) object.

But even when you add metadata for the uninstantiated generic type, calling the [Type.MakeGenericType](#) method throws a [MissingMetadataException](#) exception:

This operation cannot be carried out as metadata for the following type was removed for performance reasons:

```
App1.AppClass`1<System.Int32>.
```

You can add the following run-time directive to the runtime directives file to add `Activate` metadata for the specific instantiation over `AppClass<T>` of `System.Int32`:

XML

```
<TypeInstantiation Name="App1.AppClass" Arguments="System.Int32"
    Activate="Required Public" />
```

Each different instantiation over `AppClass<T>` requires a separate directive if it's being created with the `Type.MakeGenericType` method and not used statically.

MethodInfo.MakeGenericMethod method

Given a class `Class1` with a generic method `GetMethod<T>(T t)`, `GetMethod` can be invoked through reflection by using code like this:

C#

```
Type ct = typeof(Class1);
MethodInfo mi = ct.GetMethod("GetMethod");
Type[] typeArgs = {typeof(int)};
object[] parameters = { 12 };
var method = mi.MakeGenericMethod(typeArgs);
Class1 c = new Class1();
method.Invoke(c, parameters);
```

To run successfully, this code requires several items of metadata:

- `Browse` metadata for the type whose method you want to call.
- `Browse` metadata for the method you want to call. If it is a public method, adding public `Browse` metadata for the containing type includes the method, too.
- Dynamic metadata for the method you want to call, so that the reflection invocation delegate is not removed by the .NET Native tool chain. If dynamic metadata is missing for the method, the following exception is thrown when the `MethodInfo.MakeGenericMethod` method is called:

Output

```
MakeGenericMethod() cannot create this generic method instantiation
because the instantiation was not metadata-enabled:
'App1.Class1.GenMethod<Int32>(Int32)'.
```

The following runtime directives ensure that all required metadata is available:

XML

```
<Type Name="App1.Class1" Browse="Required PublicAndInternal">
  <MethodInstantiation Name="GenMethod" Arguments="System.Int32"
  Dynamic="Required"/>
</Type>
```

A `MethodInstantiation` directive is required for each different instantiation of the method that is dynamically invoked, and the `Arguments` element is updated to reflect each different instantiation argument.

Array.CreateInstance and Type.MakeArrayType methods

The following example calls the `Type.MakeArrayType` and `Array.CreateInstance` methods on a type, `Class1`.

C#

```
Type t = typeof(Class1);
Type at = t.MakeArrayType(1);
Array arr = Array.CreateInstance(at, 10);
```

If no array metadata is present, the following error results:

Output

This operation cannot be carried out as metadata for the following type was removed for performance reasons:

App1.Class1[]

Unfortunately, no further information is available.

`Browse` metadata for the array type is required to dynamically instantiate it. The following runtime directive allows dynamic instantiation of `Class1[]`.

XML

```
<Type Name="App1.Class1[]" Browse="Required Public" />
```

See also

- [Getting Started](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)

.NET Native Reflection API Reference

Article • 10/20/2022

.NET Native includes three new exception types:

[System.Runtime.CompilerServices.MissingInteropDataException](#),

[System.Reflection.MissingMetadataException](#), and

[System.Reflection.MissingRuntimeArtifactException](#). Note the following about all three exception types:

- These types are for internal use only.

These three exception types are for the use of the .NET Native tool chain only. The exceptions are thrown when the .NET Native tool chain detects missing data that does not allow program execution to continue.

- Do not handle these exceptions in your code.

These exceptions indicate either that metadata needed by your application is absent (the [MissingInteropDataException](#) and [MissingMetadataException](#) exceptions) or that implementation code needed by your application is missing (the [MissingRuntimeArtifactException](#) exception). You correct these exception conditions by modifying a runtime directives (.rd.xml) file to make the required metadata or implementation code available at runtime. For more information, see [Runtime Directives \(rd.xml\) Configuration File Reference](#). Two troubleshooters are available that supply the appropriate entries for your runtime directives file that will eliminate [MissingMetadataException](#) and [MissingRuntimeArtifactException](#) exceptions:

- The [MissingMetadataException troubleshooter](#) [↗](#) for types.
- The [MissingMetadataException troubleshooter](#) [↗](#) for methods.

ⓘ Note

This reference documents three exception types that are unique to .NET Native. For reference documentation for the .NET Framework core reflection API, see the [System.Reflection](#), [System.Reflection.Context](#) and [System.Reflection.Emit](#) namespaces. For reference documentation for the .NET Framework core interop API, see [System.Runtime.InteropServices](#).

System.Reflection namespace

The [System.Reflection](#) namespace contains the core types used for reflection in .NET Framework. For .NET Native, it also includes two new exception types:

Class	Description
MissingMetadataException	The exception that is thrown when reflection is used to retrieve metadata that isn't present.
MissingRuntimeArtifactException	The exception that is thrown when metadata for a type or type member is available but its implementation has been removed.

For documentation about the other types in this namespace, see [System.Reflection](#) in the .NET API reference pages.

System.Runtime.CompilerServices namespace

The [System.Runtime.CompilerServices](#) namespace includes types designed for user by language compilers. For .NET Native, it also includes a new exception type:

Class	Description
MissingInteropDataException	The exception that is thrown when a manual marshaling method is called, but metadata for a type isn't found by static analysis or in a runtime directives file.

For documentation about the other types in this namespace, see [System.Runtime.CompilerServices](#) in the .NET API reference pages.

See also

- [MissingInteropDataException Class](#)
- [MissingMetadataException Class](#)
- [MissingRuntimeArtifactException Class](#)
- [Getting Started](#)

MissingInteropDataException Class (.NET Native)

Article • 10/20/2022

.NET for Windows apps for Windows 10, .NET Native only

The exception that is thrown when a manual marshaling method is called, but metadata for a type isn't found by static analysis or in a runtime directives file.

Namespace: System.Runtime.CompilerServices

ⓘ Important

The `MissingInteropDataException` class is intended solely for internal use by the .NET Native tool chain. It is not intended for use in third-party code, nor should you handle the exception in your application code. Instead, you eliminate the exception by adding entries to your [runtime directives file](#). For more information, see the Remarks section.

Syntax

C#

```
public class MissingInteropDataException : Exception
```

The `MissingInteropDataException` class has the following members:

Constructors

Constructor	Description
<code>public MissingInteropDataException(String resourceId, Type pertinentType)</code>	Initializes a new instance of the <code>MissingInteropDataException</code> class by using the ID of a system-supplied message that describes the error and the type whose data is missing. This constructor is for internal use by the .NET Native tool chain only.

Properties

Property	Description
<code>public IDictionary Data { get; }</code>	Gets a collection of key/value pairs that provide additional user-defined information about the exception. (Inherited from System.Exception .)
<code>public string HelpLink { get; set; }</code>	Gets or sets a link to the help file associated with this exception. (Inherited from System.Exception .)
<code>public int HRESULT { get; protected set; }</code>	Gets or sets the <code>HRESULT</code> , which is a coded numeric value that is assigned to a specific exception. (Inherited from System.Exception .)
<code>public Exception InnerException { get; }</code>	Gets the exception that caused the current exception. (Inherited from System.Exception .)
<code>public string Message { get; }</code>	Gets a message that describes the current exception. (Inherited from System.Exception .)
<code>public Type MissingType { get; private set; }</code>	Gets or sets the type whose data is missing.
<code>public string Source { get; set; }</code>	Gets or sets the name of the app or object that caused the error. (Inherited from System.Exception .)
<code>public string StackTrace { get; }</code>	Gets a string representation of the immediate frames on the call stack. (Inherited from System.Exception .)
<code>public MethodBase TargetSite { get; }</code>	Gets the method that threw the current exception. (Inherited from System.Exception .)

Methods

Method	Description
<code>public bool Equals(Object obj)</code>	Determines whether the specified object is equal to the current object. (Inherited from Object .)
<code>protected void Finalize()</code>	Allows an object to try to free resources and perform other cleanup operations before it is reclaimed by garbage collection. (Inherited from Object .)
<code>public Exception GetBaseException()</code>	Returns the exception that is the root cause of one or more subsequent exceptions. (Inherited from System.Exception .)
<code>public int GetHashCode()</code>	Returns a hash code for a <code>MissingInteropDataException</code> instance. (Inherited from Object .)

Method	Description
<code>public void GetObjectData(SerializationInfo info, StreamingContext context)</code>	Sets a SerializationInfo object with information about the exception. (Inherited from System.Exception .)
<code>public Type GetType()</code>	Gets the runtime type of the current instance. (Inherited from System.Exception .)
<code>protected Object MemberwiseClone()</code>	Creates a shallow copy of the current object. (Inherited from Object .)
<code>public string ToString()</code>	Returns the string representation of the current exception. (Inherited from System.Exception .)

Events

Event	Description
<code>protected event EventHandler<SafeSerializationEventArgs> SerializeObjectState</code>	Occurs when an exception is serialized to create an exception state object that contains serialized data about the exception. (Inherited from System.Exception .)

Usage Details

The `MissingInteropDataException` exception is thrown when a method call to a COM or Windows Runtime component cannot be made successfully because type information isn't available.

The metadata that is available to an app at run time is defined by the runtime directives (XML configuration) file, *.rd.xml. To prevent your app from throwing this exception, you must modify this file to define the metadata that must be present at run time. Most commonly, you address this error by adding a `MarshalObject`, `MarshalDelegate`, or `MarshalStructure` attribute to an appropriate program element in the runtime directives file. For information about the format of this file, see [Runtime Directives \(rd.xml\) Configuration File Reference](#).

Important

Because this exception indicates that metadata needed by your application isn't available at run time, you shouldn't handle this exception in a `try/catch` block.

Instead, you should diagnose the cause of the exception and eliminate it by adding the appropriate entry to a runtime directives file.

The `MissingInteropDataException` class contains a single unique member, the `MissingType` property, that indicates the type whose metadata is needed for a successful method call. All remaining members are inherited from the base class, [System.Exception](#).

See also

- [System.Exception](#)
- [MissingMetadataException Class](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)

MissingMetadataException Class (.NET Native)

Article • 10/20/2022

.NET for Windows apps for Windows 10, .NET Native only

The exception that is thrown when reflection is used to retrieve metadata that isn't present.

Namespace: System.Reflection

Important

The `MissingMetadataException` class is intended solely for internal use by the .NET Native tool chain. It is not intended for use in third-party code, nor should you handle the exception in your application code. Instead, you eliminate the exception by adding entries to your [runtime directives file](#). For more information, see the Remarks section.

Syntax

C#

```
public sealed class MissingMetadataException : TypeAccessException
```

Note that the `MissingMetadataException` class derives from [TypeAccessException](#).

The `MissingMetadataException` class has the following members:

Constructors

Constructor	Description
<code>public MissingMetadataException()</code>	Initializes a new instance of the <code>MissingMetadataException</code> class by using a system-supplied message that describes the error. This constructor is for internal use by the .NET Native tool chain only.

Constructor	Description
<pre>public MissingMetadataException(String message)</pre>	Initializes a new instance of the <code>MissingMetadataException</code> class with a specified error message. This constructor is for internal use by the .NET Native tool chain only.

Properties

Property	Description
<pre>public IDictionary Data { get; }</pre>	Gets a collection of key/value pairs that provide additional user-defined information about the exception. (Inherited from System.Exception.)
<pre>public string HelpLink { get; set; }</pre>	Gets or sets a link to the help file associated with this exception. (Inherited from System.Exception.)
<pre>public int HRESULT { get; protected set; }</pre>	Gets or sets the <code>HRESULT</code> , a coded numeric value that is assigned to a specific exception. (Inherited from System.Exception.)
<pre>public Exception InnerException { get; }</pre>	Gets the exception that caused the current exception. (Inherited from System.Exception.)
<pre>public string Message { get; }</pre>	Gets a message that describes the current exception. (Inherited from TypeLoadException.)
<pre>public string Source { get; set; }</pre>	Gets or sets the name of the application or object that caused the error. (Inherited from System.Exception.)
<pre>public string StackTrace { get; }</pre>	Gets a string representation of the immediate frames on the call stack. (Inherited from System.Exception.)
<pre>public MethodBase TargetSite { get; }</pre>	Gets the method that threw the current exception. (Inherited from System.Exception.)
<pre>public string TypeName { get; }</pre>	Gets the fully qualified name of the type whose metadata is missing. (Inherited from TypeLoadException.)

Methods

Method	Description
--------	-------------

Method	Description
<code>public bool Equals(Object obj)</code>	Determines whether the specified object is equal to the current object. (Inherited from System.Exception .)
<code>protected void Finalize()</code>	Allows an object to try to free resources and perform other cleanup operations before it is reclaimed by garbage collection. (Inherited from Object .)
<code>public Exception GetBaseException()</code>	Returns the exception that is the root cause of one or more subsequent exceptions. (Inherited from System.Exception .)
<code>public int GetHashCode()</code>	Returns a hash code for a <code>MissingMetadataException</code> instance. (Inherited from Object .)
<code>public void GetObjectData(SerializationInfo info, StreamingContext context)</code>	Sets a SerializationInfo object with information about the exception. (Inherited from TypeLoadException .)
<code>public Type GetType()</code>	Gets the runtime type of the current instance. (Inherited from System.Exception .)
<code>protected Object MemberwiseClone()</code>	Creates a shallow copy of the current object. (Inherited from Object .)
<code>public string ToString()</code>	Returns the string representation of the current exception. (Inherited from System.Exception .)

Events

Event	Description
<code>protected event EventHandler<SafeSerializationEventArgs> SerializeObjectState</code>	Occurs when an exception is serialized to create an exception state object that contains serialized data about the exception. (Inherited from System.Exception .)

Usage Details

The `MissingMetadataException` exception is thrown when reflection is used to access metadata that isn't available in an assembly.

The metadata that is available to an app at run time is defined by the runtime directives (XML configuration) file, *.rd.xml. To prevent your app from throwing this exception, you must modify *.rd.xml to define the metadata that must be present at run time. For

information about the format of the *.rd.xml file, see [Runtime Directives \(rd.xml\) Configuration File Reference](#).

Important

Because this exception indicates that metadata needed by your application isn't available at run time, you shouldn't handle this exception in a `try/catch` block. Instead, you should diagnose the cause of the exception and eliminate it by using a runtime directives file. To get the entry that you can add to your runtime directives file that eliminates the exception, you can use one of two troubleshooters:

- The [MissingMetadataException troubleshooter](#)  for types.
- The [MissingMetadataException troubleshooter](#)  for methods.

The `MissingMetadataException` class contains no unique members; all of its members are inherited from its base class, [TypeAccessException](#).

See also

- [System.Exception](#)
- [TypeAccessException](#)
- [MissingInteropDataException Class](#)
- [MissingRuntimeArtifactException Class](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)

MissingRuntimeArtifactException Class (.NET Native)

Article • 10/20/2022

.NET for Windows apps for Windows 10, .NET Native only

The exception that is thrown when metadata for a type or type member is available but its implementation has been removed.

Namespace: System.Reflection

❗ Important

The `MissingRuntimeArtifactException` class is intended solely for internal use by the .NET Native tool chain. It is not intended for use in third-party code, nor should you handle the exception in your application code. Instead, you eliminate the exception by adding entries to your [runtime directives file](#). For more information, see the Remarks section.

Syntax

C#

```
internal sealed class MissingRuntimeArtifactException :  
    MemberAccessException
```

Note that the `MissingRuntimeArtifactException` class derives from [MemberAccessException](#).

The `MissingRuntimeArtifactException` class has the following members:

Constructors

Constructor	Description
-------------	-------------

Constructor	Description
<pre>public MissingRuntimeArtifactException()</pre>	Initializes a new instance of the <code>MissingRuntimeArtifactException</code> class by using a system-supplied message that describes the error. This constructor is for internal use by the .NET Native tool chain only.
<pre>public MissingRuntimeArtifactException(String message)</pre>	Initializes a new instance of the <code>MissingRuntimeArtifactException</code> class with a specified error message. This constructor is for internal use by the .NET Native tool chain only.

Properties

Property	Description
<pre>public IDictionary Data { get; }</pre>	Gets a collection of key/value pairs that provide additional user-defined information about the exception. (Inherited from System.Exception .)
<pre>public string HelpLink { get; set; }</pre>	Gets or sets a link to the help file associated with this exception. (Inherited from System.Exception .)
<pre>public int HRESULT { get; protected set; }</pre>	Gets or sets the <code>HRESULT</code> , a coded numeric value that is assigned to a specific exception. (Inherited from System.Exception .)
<pre>public Exception InnerException { get; }</pre>	Gets the exception that caused the current exception. (Inherited from System.Exception .)
<pre>public string Message { get; }</pre>	Gets a message that describes the current exception. (Inherited from System.Exception .)
<pre>public string Source { get; set; }</pre>	Gets or sets the name of the application or object that caused the error. (Inherited from System.Exception .)
<pre>public string StackTrace { get; }</pre>	Gets a string representation of the immediate frames on the call stack. (Inherited from System.Exception .)
<pre>public MethodBase TargetSite { get; }</pre>	Gets the method that threw the current exception. (Inherited from System.Exception .)

Methods

Method	Description
<code>public bool Equals(Object obj)</code>	Determines whether the specified object is equal to the current object. (Inherited from Object .)
<code>protected void Finalize()</code>	Allows an object to try to free resources and perform other cleanup operations before it is reclaimed by garbage collection. (Inherited from Object .)
<code>public Exception GetBaseException()</code>	Returns the exception that is the root cause of one or more subsequent exceptions. (Inherited from System.Exception .)
<code>public int GetHashCode()</code>	Returns a hash code for a <code>MissingRuntimeArtifactException</code> instance. (Inherited from Object .)
<code>public void GetObjectData(SerializationInfo info, StreamingContext context)</code>	Sets a SerializationInfo object with information about the exception. (Inherited from System.Exception .)
<code>public Type GetType()</code>	Gets the runtime type of the current instance. (Inherited from System.Exception .)
<code>protected Object MemberwiseClone()</code>	Creates a shallow copy of the current object. (Inherited from Object .)
<code>public string ToString()</code>	Returns the string representation of the current exception. (Inherited from System.Exception .)

Events

Event	Description
<code>protected event EventHandler<SafeSerializationEventArgs> SerializeObjectState</code>	Occurs when an exception is serialized to create an exception state object that contains serialized data about the exception. (Inherited from System.Exception .)

Usage Details

The `MissingRuntimeArtifactException` exception is thrown when an attempt is made to instantiate a type or invoke a type member and, although the type or member's metadata is present, its implementation has been removed.

Whether metadata and the implementation code to dynamically execute a method are available to an app at run time is defined by the runtime directives (XML configuration) file, *.rd.xml. To prevent your app from throwing this exception, you must modify *.rd.xml to ensure that the metadata needed by a type or type member is present at run time. For information about the format of the *.rd.xml file, see [Runtime Directives \(rd.xml\) Configuration File Reference](#).

Important

Because this exception indicates that the implementation code needed by your application isn't available at run time, you shouldn't handle this exception in a `try/catch` block. Instead, you should diagnose the cause of the exception and eliminate it by using a runtime directives file. Typically, you eliminate this exception by specifying the appropriate `Activate` or `Dynamic` policy for a program element in the runtime directives file (*.rd.xml file). To get the entry that you can add to your runtime directives file that eliminates the exception, you can use one of two troubleshooters:

- The [MissingMetadataException troubleshooter](#)  for types.
- The [MissingMetadataException troubleshooter](#)  for methods.

The `MissingRuntimeArtifactException` class contains no unique members; all of its members are inherited from its base class, [MemberAccessException](#).

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Policy Settings](#)

Runtime directives (rd.xml) configuration file reference

Article • 02/15/2023

A runtime directives (.rd.xml) file is an XML configuration file that specifies whether designated program elements are available for reflection. For info about how and where to add a runtime directives to your project, and how to have it recognized, see [MissingMetadataException troubleshooter](#).

Here's an example of a runtime directives file:

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Namespace Name="Contoso.Cloud.AppServices" Serialize="Required Public"
    />
    <Namespace Name="ContosoClient.ViewModels" Serialize="Required Public"
    />
    <Namespace Name="ContosoClient.DataModel" Serialize="Required Public" />
    <Namespace Name="Contoso.Reader.UtilityLib" Serialize="Required Public"
    />

    <Namespace Name="System.Collections.ObjectModel" >
      <TypeInstantiation Name="ObservableCollection"
        Arguments="ContosoClient.DataModel.ProductItem"
        Serialize="Public" />
      <TypeInstantiation Name="ReadOnlyObservableCollection"
        Arguments="ContosoClient.DataModel.ProductGroup"
        Serialize="Public" />
    </Namespace>
  </Application>
</Directives>
```

The structure of a runtime directives file

The runtime directives file uses the

`http://schemas.microsoft.com/netfx/2013/01/metadata` namespace.

The root element is the [Directives](#) element. It can contain zero or more [Library](#) elements, and zero or one [Application](#) element, as shown in the following structure. The attributes of the [Application](#) element can define application-wide runtime reflection policy, or it can serve as a container for child elements. The [Library](#) element, on the other hand, is

simply a container. The children of the [Application](#) and [Library](#) elements define the types, methods, fields, properties, and events that are available for reflection.

For reference information, choose elements from the following structure or see [Runtime Directive Elements](#). In the following hierarchy, the ellipsis marks a recursive structure. The information in brackets indicates whether that element is optional or required, and if it is used, how many instances (one or many) are allowed.

- [Directives](#) [1:1]
 - [Application](#) [0:1]
 - [Assembly](#) [0:M]
 - [Namespace](#) [0:M] . . .
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [Namespace](#) [0:M]
 - [Namespace](#) [0:M] . . .
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [Type](#) [0:M]
 - [Subtypes](#) (subclasses of the containing type) [0:1]
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [AttributeImplies](#) (containing type is an attribute) [0:1]
 - [GenericParameter](#) [0:M]
 - [Method](#) [0:M]
 - [Parameter](#) [0:M]
 - [TypeParameter](#) [0:M]
 - [GenericParameter](#) [0:M]
 - [MethodInstantiation](#) (constructed generic method) [0:M]
 - [Property](#) [0:M]
 - [Field](#) [0:M]
 - [Event](#) [0:M]
 - [TypeInstantiation](#) (constructed generic type) [0:M]
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [Method](#) [0:M]
 - [Parameter](#) [0:M]
 - [TypeParameter](#) [0:M]
 - [GenericParameter](#) [0:M]
 - [MethodInstantiation](#) (constructed generic method) [0:M]
 - [Property](#) [0:M]

- [Field](#) [0:M]
 - [Event](#) [0:M]
- [Library](#) [0:M]
 - [Assembly](#) [0:M]
 - [Namespace](#) [0:M] . . .
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [Namespace](#) [0:M]
 - [Namespace](#) [0:M] . . .
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [Type](#) [0:M]
 - [Subtypes](#) (subclasses of the containing type) [0:1]
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [AttributeImplies](#) (containing type is an attribute) [0:1]
 - [GenericParameter](#) [0:M]
 - [Method](#) [0:M]
 - [MethodInstantiation](#) (constructed generic method) [0:M]
 - [Property](#) [0:M]
 - [Field](#) [0:M]
 - [Event](#) [0:M]
 - [TypeInstantiation](#) (constructed generic type) [0:M]
 - [Type](#) [0:M] . . .
 - [TypeInstantiation](#) (constructed generic type) [0:M] . . .
 - [Method](#) [0:M]
 - [MethodInstantiation](#) (constructed generic method) [0:M]
 - [Property](#) [0:M]
 - [Field](#) [0:M]
 - [Event](#) [0:M]

The [Application](#) element can have no attributes, or it can have the policy attributes discussed in the [Runtime directive and policy section](#).

A [Library](#) element has a single attribute, `Name`, that specifies the name of a library or assembly, without its file extension. For example, the following [Library](#) element applies to an assembly named Extensions.dll.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
```

```
<!-- Child elements go here -->
</Application>
<Library Name="Extensions">
  <!-- Child elements go here -->
</Library>
</Directives>
```

Runtime directives and policy

The [Application](#) element itself and the child elements of the [Library](#) and [Application](#) elements express policy; that is, they define the way in which an app can apply reflection to a program element. The policy type is defined by an attribute of the element (for example, `Serialize`). The policy value is defined by the attribute's value (for example, `Serialize="Required"`).

Any policy specified by an attribute of an element applies to all child elements that don't specify a value for that policy. For example, if a policy is specified by a [Type](#) element, that policy applies to all contained types and members for which a policy is not explicitly specified.

The policy that can be expressed by the [Application](#), [Assembly](#), [AttributeImplies](#), [Namespace](#), [Subtypes](#), and [Type](#) elements differs from the policy that can be expressed for individual members (by the [Method](#), [Property](#), [Field](#), and [Event](#) elements).

Specifying policy for assemblies, namespaces, and types

The [Application](#), [Assembly](#), [AttributeImplies](#), [Namespace](#), [Subtypes](#), and [Type](#) elements support the following policy types:

- `Activate`. Controls runtime access to constructors, to enable activation of instances.
- `Browse`. Controls querying for information about program elements but does not enable any runtime access.
- `Dynamic`. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.
- `Serialize`. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and serialized by third-party libraries such as the Newtonsoft JSON serializer.

- `DataContractSerializer`. Controls policy for serialization that uses the [System.Runtime.Serialization.DataContractSerializer](#) class.
- `DataContractJsonSerializer`. Controls policy for JSON serialization that uses the [System.Runtime.Serialization.DataContractSerializer](#) class.
- `XmlSerializer`. Controls policy for XML serialization that uses the [System.Xml.Serialization.XmlSerializer](#) class.
- `MarshalObject`. Controls policy for marshaling reference types to WinRT and COM.
- `MarshalDelegate`. Controls policy for marshaling delegate types as function pointers to native code.
- `MarshalStructure`. Controls policy for marshaling structures to native code.

The settings associated with these policy types are:

- `All`. Enable the policy for all types and members that the tool chain does not remove.
- `Auto`. Use the default behavior. (Not specifying a policy is equivalent to setting that policy to `Auto` unless that policy is overridden, for example by a parent element.)
- `Excluded`. Disable the policy for the program element.
- `Public`. Enable the policy for public types or members unless the tool chain determines that the member is unnecessary and therefore removes it. (In the latter case, you must use `Required Public` to ensure that the member is kept and has reflection capabilities.)
- `PublicAndInternal`. Enable the policy for public and internal types or members if the tool chain doesn't remove them.
- `Required Public`. Require the tool chain to keep public types and members whether or not they are used, and enable the policy for them.
- `Required PublicAndInternal`. Require the tool chain to keep both public and internal types and members whether or not they are used, and enable the policy for them.
- `Required All`. Require the tool chain to keep all types and members whether or not they are used, and enable the policy for them.

For example, the following runtime directives file defines policy for all types and members in the assembly DataClasses.dll. It enables reflection for serialization of all public properties, enables browsing for all types and type members, enables activation for all types (because of the `Dynamic` attribute), and enables reflection for all public types and members.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="DataClasses" Serialize="Required Public"
      Browse="All" Activate="PublicAndInternal"
      Dynamic="Public" />
  </Application>
  <Library Name="UtilityLibrary">
    <!-- Child elements go here -->
  </Library>
</Directives>
```

Specifying policy for members

The `Property` and `Field` elements support the following policy types:

- `Browse` - Controls querying for information about this member but does not enable any runtime access.
- `Dynamic` - Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming. Also controls querying for information about the containing type.
- `Serialize` - Controls runtime access to the member to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer. This policy can be applied to constructors, fields, and properties.

The `Method` and `Event` elements support the following policy types:

- `Browse` - Controls querying for information about this member but doesn't enable any runtime access.
- `Dynamic` - Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming. Also controls querying for information about the containing type.

The settings associated with these policy types are:

- **Auto** - Use the default behavior. (Not specifying a policy is equivalent to setting that policy to **Auto** unless something overrides it.)
- **Excluded** - Never include metadata for the member.
- **Included** - Enable the policy if the parent type is present in the output.
- **Required** - Require the tool chain to keep this member even if appears to be unused, and enable policy for it.

Runtime directives file semantics

Policy can be defined simultaneously for both higher-level and lower-level elements. For example, policy can be defined for an assembly, and for some of the types contained in that assembly. If a particular lower-level element is not represented, it inherits the policy of its parent. For example, if an **Assembly** element is present but **Type** elements are not, the policy specified in the **Assembly** element applies to each type in the assembly.

Multiple elements can also apply policy to the same program element. For example, separate **Assembly** elements might define the same policy element for the same assembly differently. The following sections explain how the policy for a particular type is resolved in those cases.

A **Type** or **Method** element of a generic type or method applies its policy to all instantiations that do not have their own policy. For example, a **Type** element that specifies policy for **List<T>** applies to all constructed instances of that generic type, unless it's overridden for a particular constructed generic type (such as a **List<Int32>**) by a **TypeInstantiation** element. Otherwise, elements define policy for the program element named.

When an element is ambiguous, the engine looks for matches, and if it finds an exact match, it will use it. If it finds multiple matches, there will be a warning or error.

If two directives apply policy to the same program element

If two elements in different runtime directives files try to set the same policy type for the same program element (such as an assembly or type) to different values, the conflict is resolved as follows:

1. If the **Excluded** element is present, it has precedence.
2. **Required** has precedence over not **Required**.

3. `All` has precedence over `PublicAndInternal`, which has precedence over `Public`.

4. Any explicit setting has precedence over `Auto`.

For example, if a single project includes the following two runtime directives files, the serialization policy for `DataClasses.dll` is set to both `Required Public` and `All`. In this case, the serialization policy would be resolved as `Required All`.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="DataClasses" Serialize="Required Public"/>
  </Application>
  <Library Name="DataClasses">
    <!-- any other elements -->
  </Library>
</Directives>
```

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="DataClasses" Serialize="All" />
  </Application>
  <Library Name="DataClasses">
    <!-- any other elements -->
  </Library>
</Directives>
```

However, if two directives in a single runtime directives file try to set the same policy type for the same program element, the XML Scheme Definition tool displays an error message.

If child and parent elements apply the same policy type

Child elements override their parent elements, including the `Excluded` setting.

Overriding is the main reason you would want to specify `Auto`.

In the following example, the serialization policy setting for everything in `DataClasses` that's not in `DataClasses.ViewModels` would be `Required Public`, and everything that's in both `DataClasses` and `DataClasses.ViewModels` would be `All`.

XML

```

<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="DataClasses" Serialize="Required Public" >
      <Namespace Name="DataClasses.ViewModels" Serialize="All" />
    </Assembly>
  </Application>
  <Library Name="DataClasses">
    <!-- any other elements -->
  </Library>
</Directives>

```

If open generics and instantiated elements apply the same policy type

In the following example, `Dictionary<int,int>` is assigned the `Browse` policy only if the engine has another reason to give it the `Browse` policy (which would otherwise be the default behavior); every other instantiation of `Dictionary<TKey,TValue>` will have all of its members browsable.

XML

```

<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="DataClasses" Serialize="Required Public" >
      <Namespace Name="DataClasses.ViewModels" Serialize="All" />
    </Assembly>
    <Namespace Name="DataClasses.Generics" />
    <Type Name="Dictionary" Browse="All" />
    <TypeInstantiation Name="Dictionary"
      Arguments="System.Int32,System.Int32" Browse="Auto"
    />
  </Application>
  <Library Name="DataClasses">
    <!-- any other elements -->
  </Library>
</Directives>

```

How policy is inferred

Each policy type has a different set of rules that determine how the presence of that policy type affects other constructs.

The effect of Browse policy

Applying the `Browse` policy to a type involves the following policy changes:

- The base type of the type is marked with the `Browse` policy.
- If the type is an instantiated generic, the uninstantiated version of the type is marked with the `Browse` policy.
- If the type is a delegate, the `Invoke` method on the type is marked with the `Dynamic` policy.
- Each interface of the type is marked with the `Browse` policy.
- The type of each attribute applied to the type is marked with the `Browse` policy.
- If the type is generic, each constraint type is marked with the `Browse` policy.
- If the type is generic, the types over which the type is instantiated are marked with the `Browse` policy.

Applying the `Browse` policy to a method involves the following policy changes:

- Each parameter type of the method is marked with the `Browse` policy.
- The return type of the method is marked with the `Browse` policy.
- The containing type of the method is marked with the `Browse` policy.
- If the method is an instantiated generic method, the uninstantiated generic method is marked with the `Browse` policy.
- The type of each attribute applied to the method is marked with the `Browse` policy.
- If the method is generic, each constraint type is marked with the `Browse` policy.
- If the method is generic, the types over which the method is instantiated are marked with the `Browse` policy.

Applying the `Browse` policy to a field involves the following policy changes:

- The type of each attribute applied to the field is marked with the `Browse` policy.
- The type of the field is marked with the `Browse` policy.
- The type to which the field belongs is marked with the `Browse` policy.

The effect of Dynamic policy

Applying the `Dynamic` policy to a type involves the following policy changes:

- The base type of the type is marked with the `Dynamic` policy.
- If the type is an instantiated generic, the uninstantiated version of the type is marked with the `Dynamic` policy.
- If the type is a delegate type, the `Invoke` method on the type is marked with the `Dynamic` policy.
- Each interface of the type is marked with the `Browse` policy.
- The type of each attribute applied to the type is marked with the `Browse` policy.
- If the type is generic, each constraint type is marked with the `Browse` policy.
- If the type is generic, the types over which the type is instantiated are marked with the `Browse` policy.

Applying the `Dynamic` policy to a method involves the following policy changes:

- Each parameter type of the method is marked with the `Browse` policy.
- The return type of the method is marked with the `Dynamic` policy.
- The containing type of the method is marked with the `Dynamic` policy.
- If the method is an instantiated generic method, the uninstantiated generic method is marked with the `Browse` policy.
- The type of each attribute applied to the method is marked with the `Browse` policy.
- If the method is generic, each constraint type is marked with the `Browse` policy.
- If the method is generic, the types over which the method is instantiated are marked with the `Browse` policy.
- The method can be invoked by `MethodInfo.Invoke`, and delegate creation becomes possible by [MethodInfo.CreateDelegate](#).

Applying the `Dynamic` policy to a field involves the following policy changes:

- The type of each attribute applied to the field is marked with the `Browse` policy.
- The type of the field is marked with the `Dynamic` policy.
- The type to which the field belongs is marked with the `Dynamic` policy.

The effect of Activation policy

Applying the Activation policy to a type involves the following policy changes:

- If the type is an instantiated generic, the uninstantiated version of the type is marked with the `Browse` policy.
- If the type is a delegate type, the `Invoke` method on the type is marked with the `Dynamic` policy.
- Constructors of the type are marked with the `Activation` policy.

Applying the `Activation` policy to a method involves the following policy change:

- The constructor can be invoked by the `ConstructorInfo.Invoke` and `Activator.CreateInstance` methods. For methods, the `Activation` policy affects constructors only.

Applying the `Activation` policy to a field has no effect.

The effect of Serialize policy

The `Serialize` policy enables the use of common reflection-based serializers. However, because the exact reflection access patterns of non-Microsoft serializers are not known to Microsoft, this policy may not be entirely effective.

Applying the `Serialize` policy to a type involves the following policy changes:

- The base type of the type is marked with the `Serialize` policy.
- If the type is an instantiated generic, the uninstantiated version of the type is marked with the `Browse` policy.
- If the type is a delegate type, the `Invoke` method on the type is marked with the `Dynamic` policy.
- If the type is an enumeration, an array of the type is marked with the `Serialize` policy.
- If the type implements `IEnumerable<T>`, `T` is marked with the `Serialize` policy.
- If the type is `IEnumerable<T>`, `IList<T>`, `ICollection<T>`, `ReadOnlyCollection<T>`, or `ReadOnlyList<T>`, then `T[]` and `List<T>` are marked with the `Serialize` policy, but no members of the interface type are marked with the `Serialize` policy.

- If the type is `List<T>`, no members of the type are marked with the `Serialize` policy.
- If the type is `IDictionary<TKey,TValue>`, `Dictionary<TKey,TValue>` is marked with the `Serialize` policy. but no members of the type are marked with the `Serialize` policy.
- If the type is `Dictionary<TKey,TValue>`, no members of the type are marked with the `Serialize` policy.
- If the type implements `IDictionary<TKey,TValue>`, `TKey` and `TValue` are marked with the `Serialize` policy.
- Each constructor, each property accessor, and each field is marked with the `Serialize` policy.

Applying the `Serialize` policy to a method involves the following policy changes:

- The containing type is marked with the `Serialize` policy.
- The return type of the method is marked with the `Serialize` policy.

Applying the `Serialize` policy to a field involves the following policy changes:

- The containing type is marked with the `Serialize` policy.
- The type of the field is marked with the `Serialize` policy.

The effect of `XmlSerializer`, `DataContractSerializer`, and `DataContractJsonSerializer` policies

Unlike the `Serialize` policy, which is intended for reflection-based serializers, the `XmlSerializer`, `DataContractSerializer`, and `DataContractJsonSerializer` policies are used to enable a set of serializers that are known to the .NET Native tool chain. These serializers are not implemented by using reflection, but the set of types that can be serialized at run time is determined in a similar manner as types that are reflectable.

Applying one of these policies to a type enables the type to be serialized with the matching serializer. Also, any types that the serialization engine can statically determine as needing serialization will also be serializable.

These policies have no effect on methods or fields.

For more information, see the "Differences in Serializers" section in [Migrating Your Windows Store App to .NET Native](#).

See also

- [Runtime Directive Elements](#)
- [Reflection and .NET Native](#)